```
# Q1. Student Performance DataFrame: Construction and Analysis
# A college wants to analyze student performance using the following data.
import pandas as pd
import numpy as np
data = {    "Student_ID": [        101, 102, 103, 104, 105, 106,        107, 108, 109, 110, 111, 112    ],
        "Name": [  "Amit", "Priya", "Rahul", "Neha",        "Vikas", "Pooja", "Karan", "Simran",  "Arjun", "Meena", "Rohit'
            "Age": [        19, 20, 18, 21, 20, 19,        22, 18, 21, 20, 19, 22    ],
            "Hours_Studied": [        2, 5, 3, 8, 6, 4,        9, 2, 7, 5, 6, 8    ],
            "Attendance": [ 65, 85, 70, 95, 88, 75,        98, 60, 90, 82, 86, 94 ],
        "Marks": [48, 78, 55, 92, 84, 67,        96, 45, 89, 76, 81, 93 ]
}
```

```
df=pd.DataFrame(data)
```

```
df
```

	Student_ID	Name	Age	Hours_Studied	Attendance	Marks
0	101	Amit	19	2	65	48
1	102	Priya	20	5	85	78
2	103	Rahul	18	3	70	55
3	104	Neha	21	8	95	92
4	105	Vikas	20	6	88	84
5	106	Pooja	19	4	75	67
6	107	Karan	22	9	98	96
7	108	Simran	18	2	60	45
8	109	Arjun	21	7	90	89
9	110	Meena	20	5	82	76
10	111	Rohit	19	6	86	81
11	112	Anjali	22	8	94	93

Next steps: Generate code with df | New interactive sheet

```
# 1.Create a DataFrame and display its first five and last three records.
df.head(5)
df.tail(3)
```

	Student_ID	Name	Age	Hours_Studied	Attendance	Marks
9	110	Meena	20	5	82	76
10	111	Rohit	19	6	86	81
11	112	Anjali	22	8	94	93

```
# 2.Determine the number of rows and columns programmatically. Verify that the result matches the expected dimensions.
df.shape
```

```
(12, 6)
```

```
df.shape == (12, 6)
print(type(df))
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
# 3.Extract all column names as a Python list and classify them as identifiers, categorical attributes, or numerical attrit
# Extract all column names as a list
columns = df.columns.tolist()
print("All Columns:", columns)

identifiers = ["Student_ID"]

categorical_attributes = ["Name"]

numerical_attributes = ["Age", "Hours_Studied", "Attendance", "Marks"]

print("Identifiers:", identifiers)
```

```
print("Categorical Attributes:", categorical_attributes)
print("Numerical Attributes:", numerical_attributes)
```

```
All Columns: ['Student_ID', 'Name', 'Age', 'Hours_Studied', 'Attendance', 'Marks']
Identifiers: ['Student_ID']
Categorical Attributes: ['Name']
Numerical Attributes: ['Age', 'Hours_Studied', 'Attendance', 'Marks']
```

```
# 4 . Select the `Name`, `Hours_Studied`, and `Marks` columns. Then identify students whose marks are above the dataset's av

df1 = df[["Name", "Hours_Studied", "Marks"]]

avg_marks = df["Marks"].mean()

print("Average Marks:", avg_marks)


result = df1[df1["Marks"] > avg_marks]

print(result)
```

```
Average Marks: 75.33333333333333
      Name  Hours_Studied  Marks
1    Priya              5     78
3     Neha              8     92
4    Vikas              6     84
6    Karan              9     96
8    Arjun              7     89
9    Meena              5     76
10   Rohit              6     81
11  Anjali              8     93
```

```
# 5.Reset the index after filtering, compare the original and new indices, and explain why an index should not automatical
# Select required columns
df1 = df[["Name", "Hours_Studied", "Marks"]]

avg_marks = df["Marks"].mean()
result = df1[df1["Marks"] > avg_marks]


print("Original Index:")
print(result)

new_result = result.reset_index(drop=True)


print("\nNew Index:")
print(new_result)
```

```
Original Index:
      Name  Hours_Studied  Marks
1    Priya              5     78
3     Neha              8     92
4    Vikas              6     84
6    Karan              9     96
8    Arjun              7     89
9    Meena              5     76
10   Rohit              6     81
11  Anjali              8     93

New Index:
     Name  Hours_Studied  Marks
0   Priya              5     78
1    Neha              8     92
2   Vikas              6     84
3   Karan              9     96
4   Arjun              7     89
5   Meena              5     76
6   Rohit              6     81
7  Anjali              8     93
```

Q2. Advanced Row and Column Selection Using iloc and loc

```
# 1.Select rows at positions 2 through 7 using iloc, returning only Name, Attendance, and Marks. Explain why the ending pos
# Select rows from position 2 to 7
result = df.iloc[2:8][["Name", "Attendance", "Marks"]]

print(result)
```

```
     Name  Attendance  Marks
2   Rahul          70     55
3    Neha          95     92
4   Vikas          88     84
```

```
5   Pooja          75     67
6   Karan          98     96
7  Simran          60     45
```

```
#2. Create a copy of the DataFrame and set Student_ID as its index. Use loc to retrieve students with IDs 103, 107, and 11

df_copy = df.copy()

df_copy.set_index("Student_ID", inplace=True)


result = df_copy.loc[[103, 107, 110]]

print(result)
```

```
             Name  Age  Hours_Studied  Attendance  Marks
Student_ID
103         Rahul   18              3          70     55
107         Karan   22              9          98     96
110         Meena   20              5          82     76
```

```
# 3.Using Boolean conditions, identify students who studied at least 6 hours, have attendance of at least 85%, and scored m
result = df[
    (df["Hours_Studied"] >= 6) &
    (df["Attendance"] >= 85) &
    (df["Marks"] > 80)
]

print(result)
```

```
    Student_ID    Name  Age  Hours_Studied  Attendance  Marks
3          104    Neha   21              8          95     92
4          105   Vikas   20              6          88     84
6          107   Karan   22              9          98     96
8          109   Arjun   21              7          90     89
10         111   Rohit   19              6          86     81
11         112  Anjali   22              8          94     93
```

```
# 4.Using iloc, extract the last four students and the last three columns without writing any column names explicitly.
result = df.iloc[-4:, -3:]

print(result)
```

```
    Hours_Studied  Attendance  Marks
8               7          90     89
9               5          82     76
10              6          86     81
11              8          94     93
```

```
#5. Create a new DataFrame containing students whose marks are below the median. Preserve their original student IDs and ex

median_marks = df["Marks"].median()

result = df[df["Marks"] < median_marks].copy()

result.set_index("Student_ID", inplace=True)

print("Median Marks:", median_marks)
print(result)
```

```
Median Marks: 79.5
             Name  Age  Hours_Studied  Attendance  Marks
Student_ID
101          Amit   19              2          65     48
102         Priya   20              5          85     78
103         Rahul   18              3          70     55
106         Pooja   19              4          75     67
108        Simran   18              2          60     45
110         Meena   20              5          82     76
```

Q3. Series vs. DataFrame: Feature Selection and Validation

```
import pandas as pd
df = pd.DataFrame({    "Student_ID": [201, 202, 203, 204, 205,   206, 207, 208, 209, 210    ],

 "Hours_Studied": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10    ],
  "Attendance": [50, 55, 60, 65, 70, 75, 80, 85, 90, 95    ],
 "Sleep_Hours": [  8, 7, 8, 6, 7, 6, 7, 8, 6, 7    ],
  "Marks":  [35, 40, 48, 52, 61, 68, 74, 82, 90, 96  ]
       }
 )
```

```
df
```

	Student_ID	Hours_Studied	Attendance	Sleep_Hours	Marks
0	201	1	50	8	35
1	202	2	55	7	40
2	203	3	60	8	48
3	204	4	65	6	52
4	205	5	70	7	61
5	206	6	75	6	68
6	207	7	80	7	74
7	208	8	85	8	82
8	209	9	90	6	90
9	210	10	95	7	96

Next steps: Generate code with df | New interactive sheet

```
#1.. Create X_single using only Hours_Studied and y using Marks. Demonstrate the difference between selecting the feature v

X_single = df["Hours_Studied"]   # Single feature

X_single_2d = df[["Hours_Studied"]]   # Double square brackets
y = df["Marks"]     # Target variable

print("Single brackets:")
print(type(X_single))
print(X_single.shape)

print("\nDouble brackets:")
print(type(X_single_2d))
print(X_single_2d.shape)

print("\nTarget:")
print(type(y))
print(y.shape)
```

```
# 2..Create X_multi using Hours_Studied, Attendance, and Sleep_Hours. Verify that Marks and Student_ID are excluded from th
X_multi = df[["Hours_Studied", "Attendance", "Sleep_Hours"]]
y = df["Marks"]

print("X_multi:")
print(X_multi)

print("\nFeature columns:", X_multi.columns.tolist())
print("Shape:", X_multi.shape)

print("\nMarks included:", "Marks" in X_multi.columns)
print("Student_ID included:", "Student_ID" in X_multi.columns)
```

```
X_multi:
   Hours_Studied  Attendance  Sleep_Hours
0              1          50            8
1              2          55            7
2              3          60            8
3              4          65            6
4              5          70            7
5              6          75            6
6              7          80            7
7              8          85            8
8              9          90            6
9             10          95            7

Feature columns: ['Hours_Studied', 'Attendance', 'Sleep_Hours']
Shape: (10, 3)

Marks included: False
Student_ID included: False
```

```
# 3..Use drop() to construct features and target variables. Verify programmatically that no target column remains in X.

X = df.drop(columns=["Marks", "Student_ID"])
```

```
y = df["Marks"]

print("X columns:", X.columns.tolist())
print("y name:", y.name)


print("Marks present in X:", "Marks" in X.columns)
print("Student_ID present in X:", "Student_ID" in X.columns)
```

```
X columns: ['Hours_Studied', 'Attendance', 'Sleep_Hours']
y name: Marks
Marks present in X: False
Student_ID present in X: False
```

```
# 4..Write validation code that checks whether X and y contain the same number of observations, whether X has the intended
X = df.drop(columns=["Marks", "Student_ID"])
y = df["Marks"]

print("Same number of observations:", len(X) == len(y))
print("Correct number of features:", X.shape[1] == 3)
print("Target is one-dimensional:", y.ndim == 1)


print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
Same number of observations: True
Correct number of features: True
Target is one-dimensional: True
X shape: (10, 3)
y shape: (10,)
```

```
# 5..Explain why including Student_ID as a predictive feature may be inappropriate, even though it contains numerical value
X_good = df[["Hours_Studied", "Attendance", "Sleep_Hours"]]


X_with_id = df[[
    "Student_ID",
    "Hours_Studied",
    "Attendance",
    "Sleep_Hours"
]]

print("Recommended features:")
print(X_good.head())

print("\nFeatures including Student_ID:")
print(X_with_id.head())

print("\nRecommended shape:", X_good.shape)
print("Shape including ID:", X_with_id.shape)
```

Q4. End-to-End Data Preparation Challenge:: A training institute wants to predict students' final examination marks using their study behavior and attendance.

```
import pandas as pd
df = pd.DataFrame({    "Student_ID": range(401, 421),
  "Name": [ "A", "B", "C", "D", "E", "F", "G", "H", "I", "J", "K", "L", "M", "N", "O", "P", "Q", "R", "S", "T"    ],
"Hours_Studied": [ 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 2, 4, 6, 8, 3, 5, 7, 9, 1, 10 ],
"Attendance": [50, 55, 60, 65, 70, 75, 80, 85, 90, 95,58, 67, 78, 88, 62, 73, 83, 92, 52, 98 ],
 "Sleep_Hours": [ 5, 6, 7, 8, 6, 7, 8, 5, 6, 7,  8, 5, 6, 7, 8, 6, 7, 5, 6, 8    ],
"Final_Marks": [        30, 38, 45, 52, 60, 67, 75, 83, 90, 98, 40, 54, 70, 87, 48, 63, 78, 92, 34, 96    ]})
```

```
# 1..Inspect the DataFrame and write a reusable function that reports the number of rows, columns, column names, data types
def inspect_dataframe(df):
    print("Number of Rows:", df.shape[0])
    print("Number of Columns:", df.shape[1])

    print("\nColumn Names:")
    print(df.columns.tolist())

    print("\nData Types:")
    print(df.dtypes)

    print("\nMissing Values:")
    print(df.isnull().sum())


inspect_dataframe(df)
```

```
# 2..Build two feature sets: one using only Hours_Studied, and another using Hours_Studied, Attendance, and Sleep_Hours. Us
# Single-feature dataset
X_single = df[["Hours_Studied"]]

# Multi-feature dataset
X_multi = df[["Hours_Studied", "Attendance", "Sleep_Hours"]]

# Target variable
y = df["Final_Marks"]

print("Single-feature shape:", X_single.shape)
print("Multi-feature shape:", X_multi.shape)
print("Target shape:", y.shape)

print("\nSingle-feature columns:", X_single.columns.tolist())
print("Multi-feature columns:", X_multi.columns.tolist())
```

```
Single-feature shape: (20, 1)
Multi-feature shape: (20, 3)
Target shape: (20,)

Single-feature columns: ['Hours_Studied']
Multi-feature columns: ['Hours_Studied', 'Attendance', 'Sleep_Hours']
```

```
# 3..Filter students with attendance below 70, then select their names, IDs, and marks. Preserve the student IDs after filt
result = df[df["Attendance"] < 70][
    ["Name", "Student_ID", "Final_Marks"]
]

print(result)
```

	Name	Student_ID	Final_Marks
0	A	401	30
1	B	402	38
2	C	403	45
3	D	404	52
10	K	411	40
11	L	412	54
14	O	415	48
18	S	419	34

```
#4.. Create a reproducible 20% test split for the multi-feature dataset. Verify that the training and testing indices do not
from sklearn.model_selection import train_test_split

X = df[["Hours_Studied", "Attendance", "Sleep_Hours"]]
y = df["Final_Marks"]

train_idx, test_idx = train_test_split(
    df.index,
    test_size=0.20,
    random_state=42
)

X_train = X.loc[train_idx]
X_test = X.loc[test_idx]

y_train = y.loc[train_idx]
y_test = y.loc[test_idx]

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)

print("\nIndices overlap:", len(set(train_idx) & set(test_idx)) > 0)

# Verify all observations are accounted for
print("Total observations:", len(X_train) + len(X_test))
```

```
X_train shape: (16, 3)
X_test shape: (4, 3)
y_train shape: (16,)
y_test shape: (4,)

Indices overlap: False
Total observations: 20
```

```
# 5..Develop a complete data-preparation pipeline in Python that creates the DataFrame, validates the feature and target col
from sklearn.model_selection import train_test_split

df = pd.DataFrame({
    "Student_ID": range(401, 421),
    "Name": list("ABCDEFGHIJKLMNOPQRST")
```

```
    "Name": list("ABCDEFGHIJKLMNOPQRST"),
    "Hours_Studied": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10,
                      2, 4, 6, 8, 3, 5, 7, 9, 1, 10],
    "Attendance": [50, 55, 60, 65, 70, 75, 80, 85, 90, 95,
                   58, 67, 78, 88, 62, 73, 83, 92, 52, 98],
    "Sleep_Hours": [5, 6, 7, 8, 6, 7, 8, 5, 6, 7,
                    8, 5, 6, 7, 8, 6, 7, 5, 6, 8],
    "Final_Marks": [30, 38, 45, 52, 60, 67, 75, 83, 90, 98,
                    40, 54, 70, 87, 48, 63, 78, 92, 34, 96]
})

feature_cols = [
    "Hours_Studied",
    "Attendance",
    "Sleep_Hours"
]
target_col = "Final_Marks"

required_cols = feature_cols + [target_col]

assert all(col in df.columns for col in required_cols), \
    "Required columns are missing!"

assert not df[required_cols].isnull().any().any(), \
    "Missing values found in features or target!"

X = df[feature_cols]
y = df[target_col]


assert len(X) == len(y), "X and y have different row counts!"
assert X.shape[1] == 3, "Unexpected number of features!"
assert y.ndim == 1, "Target must be one-dimensional!"


X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42
)
print("DATA PREPARATION REPORT")
print("-" * 30)
print("Original DataFrame:", df.shape)
print("Features:", X.columns.tolist())
print("Target:", y.name)
print("Training features:", X_train.shape)
print("Testing features:", X_test.shape)
print("Training target:", y_train.shape)
print("Testing target:", y_test.shape)
print("Missing values:", df[required_cols].isnull().sum().sum())
print("Index overlap:", not set(X_train.index).isdisjoint(X_test.index))
print("Pipeline validation: Passed")
```